

DDL commands:

```
CREATE TABLE Users (  
    userId INT PRIMARY KEY,  
    email VARCHAR(255),  
    password VARCHAR(255));
```

```
CREATE TABLE Crops (  
    cropId INT PRIMARY KEY,  
    cropName VARCHAR(255));
```

```
CREATE TABLE Countries (  
    countryId INT PRIMARY KEY,  
    countryName VARCHAR(255));
```

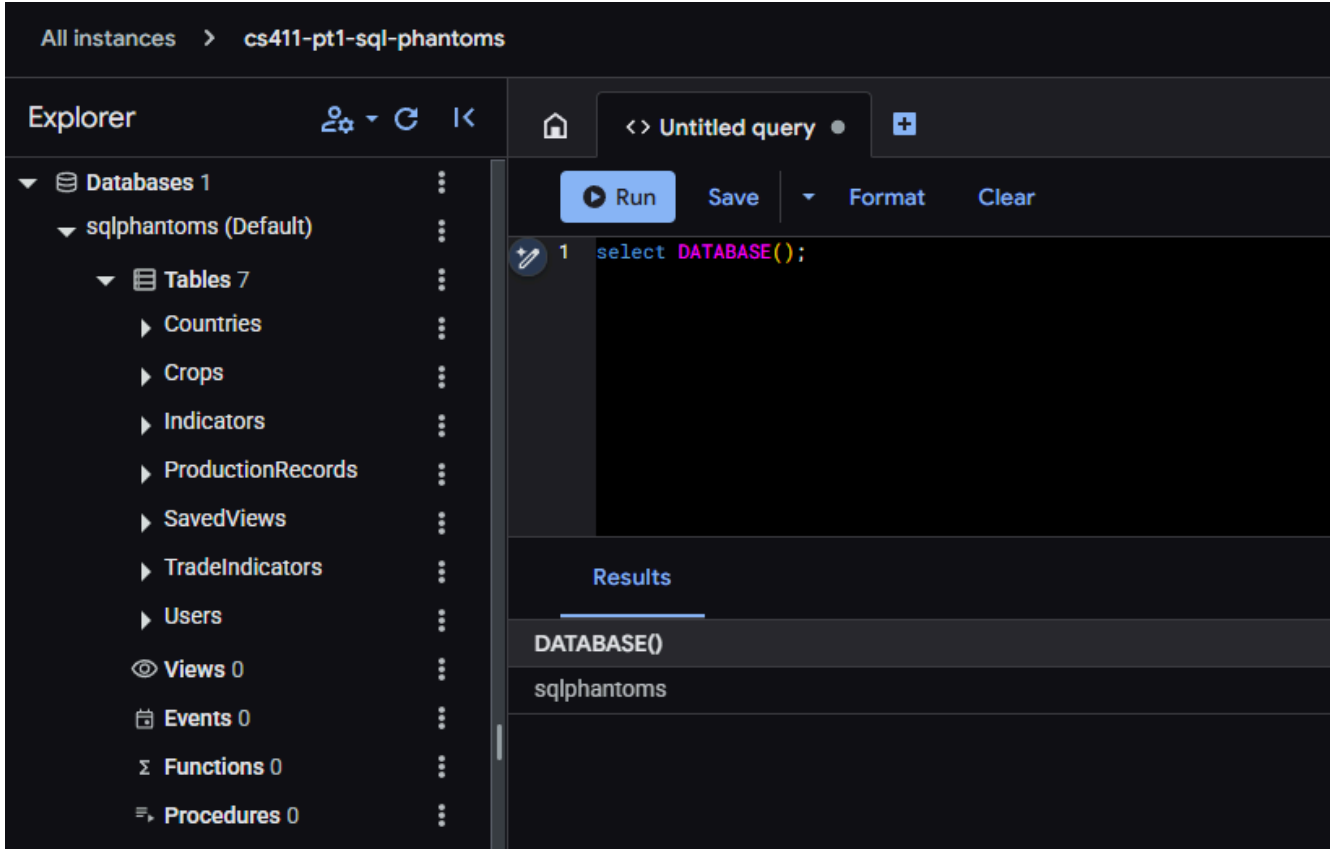
```
CREATE TABLE ProductionRecords (  
    recordId INT PRIMARY KEY,  
    countryId INT,  
    cropId INT,  
    element VARCHAR(255),  
    unit VARCHAR(20),  
    value REAL,  
    year INT,  
    FOREIGN KEY (countryId) REFERENCES Countries(countryId),  
    FOREIGN KEY (cropId) REFERENCES Crops(cropId));
```

```
CREATE TABLE Indicators (  
    indicatorId INT PRIMARY KEY,  
    indicatorName VARCHAR(255));
```

```
CREATE TABLE TradeIndicators (  
    indicatorRecordId INT PRIMARY KEY,  
    countryId INT,  
    indicatorId INT,  
    year INT,  
    value REAL,  
    FOREIGN KEY (countryId) REFERENCES Countries(countryId),  
    FOREIGN KEY (indicatorId) REFERENCES Indicators(indicatorId));
```

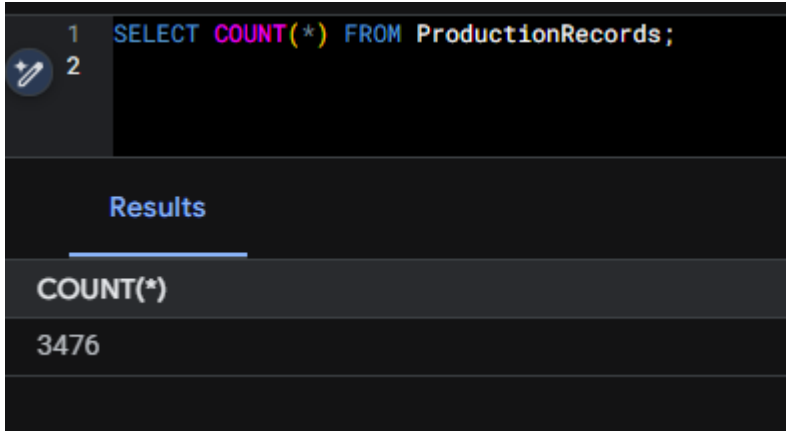
```
CREATE TABLE SavedViews (  
    viewId INT PRIMARY KEY,  
    userId INT,  
    title VARCHAR(255),  
    filters JSON,  
    FOREIGN KEY (userId) REFERENCES Users(userId));
```

Connection:



Row counts:

ProductionRecords table:



Users table:

	1	<code>SELECT COUNT(*) FROM Users;</code>	
	2		
Results			
COUNT(*)			
1001			

SaveViews table:

	1	<code>SELECT COUNT(*) FROM SavedViews;</code>	
	2		
Results			
COUNT(*)			
1030			

Query 1:

This query returns the countries that are performing better than average in crop production and trade.

```
SELECT co.countryId, co.countryName, 'Top Producer' AS reason
FROM Countries co
JOIN ProductionRecords pr ON pr.countryId = co.countryId
GROUP BY co.countryId, co.countryName
HAVING SUM(pr.value) > (
    SELECT AVG(countryTotal.total)
    FROM (
        SELECT countryId, SUM(value) AS total
        FROM ProductionRecords
        GROUP BY countryId
    ) AS countryTotal
)
UNION
SELECT co.countryId, co.countryName, 'Top Trade Activity' AS reason
FROM Countries co
JOIN TradeIndicators ti ON ti.countryId = co.countryId
GROUP BY co.countryId, co.countryName
HAVING SUM(ti.value) > (
    SELECT AVG(countryTotal.total)
    FROM (
        SELECT countryId, SUM(value) AS total
        FROM TradeIndicators
        GROUP BY countryId
    ) AS countryTotal
)
ORDER BY countryName;
```

Output: The output is only 9 rows as the number of countries are limited to 16 countries from the Middle East region.

countryId	countryName	reason
2	Egypt	Top Producer
2	Egypt	Top Trade Activity
3	Iran	Top Producer
3	Iran	Top Trade Activity
5	Israel	Top Trade Activity
9	Oman	Top Trade Activity
13	Syria	Top Trade Activity
14	Turkey	Top Producer
14	Turkey	Top Trade Activity

Query 2:

This query supplies coordinates for dynamic line graphs and scatter plots comparing production vs. import metrics.

```
SELECT
  co.countryName,
  cr.cropName,
  pr.element,
  pr.year,
  SUM(pr.value) AS annual_aggregated_value,
  (SELECT AVG(ti.value)
   FROM TradeIndicators ti
   WHERE ti.countryId = co.countryId
        AND ti.year = pr.year
  ) AS baseline_macro_index
FROM Countries co
JOIN ProductionRecords pr ON co.countryId = pr.countryId
JOIN Crops cr ON pr.cropId = cr.cropId
GROUP BY co.countryId, co.countryName, cr.cropName, pr.element, pr.year
ORDER BY annual_aggregated_value DESC
```

Output:

countryName	cropName	element	year	annual_aggregated_value	baseline_macro_index
Egypt	Potatoes	Production (tonnes)	2023	8.572519e+06	10.6003111645036
Egypt	Potatoes	Production (tonnes)	2024	8.08229082e+06	13.7121128792805
Egypt	Potatoes	Production (tonnes)	2022	7.210464e+06	10.9457535224737
Turkey	Potatoes	Production (tonnes)	2024	6.9e+06	5.82042467876951
Egypt	Potatoes	Production (tonnes)	2020	6.78634e+06	11.1668239443487
Egypt	Potatoes	Production (tonnes)	2021	6.273931e+06	11.4369326589725
Turkey	Potatoes	Production (tonnes)	1999	6e+06	9.061961123945675
Turkey	Potatoes	Production (tonnes)	2023	5.7e+06	9.428726243876035
Turkey	Potatoes	Production (tonnes)	2000	5.37e+06	8.82968849108873
Turkey	Potatoes	Production (tonnes)	2003	5.3e+06	9.200189688575534
Turkey	Potatoes	Production (tonnes)	1998	5.25e+06	10.098326594563035
Egypt	Potatoes	Production (tonnes)	2019	5.200563e+06	10.6971354538956
Turkey	Potatoes	Production (tonnes)	2002	5.2e+06	9.340901941758396
Turkey	Potatoes	Production (tonnes)	2020	5.2e+06	9.295409045857124
Turkev	Potatoes	Production (tonnes)	2022	5.2e+06	9.41028368029172

Rows per page: 20 1 – 20 of 3476 |< < > >|

Query 3:

The query identifies the highest-production year for each country and connects that production result with trade indicator data from the same country and year.

```
WITH country_year_summary AS (  
    SELECT co.countryId, co.countryName, pr.year, SUM(pr.value) AS  
total_production, AVG(ti.value) AS avg_trade_indicator, COUNT(DISTINCT pr.cropId) AS  
number_of_crops_reported  
    FROM Countries co  
    JOIN ProductionRecords pr ON co.countryId = pr.countryId  
    JOIN TradeIndicators ti ON co.countryId = ti.countryId AND pr.year = ti.year  
    WHERE pr.element LIKE '%Production%'  
    GROUP BY co.countryId, co.countryName, pr.year  
) ,  
ranked_country_years AS (  
    SELECT countryName, year, total_production, avg_trade_indicator, number_of_crops_reported,  
        ROW_NUMBER() OVER (  
            PARTITION BY countryName  
            ORDER BY total_production DESC  
        ) AS production_rank  
    FROM country_year_summary  
)  
SELECT countryName, year, total_production, avg_trade_indicator, number_of_crops_reported,  
    CASE  
        WHEN total_production > (  
            SELECT AVG(total_production)  
            FROM country_year_summary  
        )  
        THEN 'Above Regional Average Production'  
        ELSE 'Below Regional Average Production'  
    END AS production_status  
FROM ranked_country_years  
WHERE production_rank = 1  
ORDER BY total_production DESC  
LIMIT 15;
```

Output:

countryName	year	total_production	avg_trade_indicator	number_of_crops_reported	production_status
Turkey	2023	1.8436054e+07	9.428726243876035	3	Above Regional Average Production
Iran	2016	1.4105804e+07	12.513751697089816	3	Above Regional Average Production
Egypt	2023	1.1816687e+07	10.6003111645036	4	Above Regional Average Production
Syria	2008	1.967184e+06	14.256601315019898	3	Below Regional Average Production
Israel	2014	1.344348e+06	19.934816387439852	4	Below Regional Average Production
Saudi Arabia	2023	1.24350256e+06	1.4469719374237369	1	Below Regional Average Production
Iraq	2002	993776	8.5626717122094	3	Below Regional Average Production
Lebanon	2022	834527	1.1895732629522	2	Below Regional Average Production
Jordan	2016	551858	7.324316886142055	2	Below Regional Average Production
Yemen	2010	368860.55	9.87370902755356	3	Below Regional Average Production
Palestine	2013	178359.02	5.62595068037323	3	Below Regional Average Production
Kuwait	2011	66407	0.438835825324165	1	Below Regional Average Production
Oman	2015	46288	3.75086366724222	1	Below Regional Average Production
United Arab Emirates	2000	15578	2.26254149328885	1	Below Regional Average Production
Qatar	2023	2048.8	8.930173349408706	1	Below Regional Average Production
Rows per page:					20 ▾ 1 – 15 of 15 < < > >

Part 2: Indexing

Query 1 with index on year:

```
EXPLAIN
-> Sort: countryName (cost=2.6..2.6 rows=0) (actual time=12.8..12.8 rows=9 loops=1) -> Table scan on <union temporary> (cost=2.5..2.5 rows=0) (actual time=12.8..12.8 rows=9 loops=1) -> Union materialize with deduplication (cost=0..0 rows=0) (actual time=12.8..12.8 rows=9 loops=1) -> Filter: ('sum(pr.`value`)' > (select #2)) (actual time=10.2..10.2 rows=3 loops=1) -> Table scan on <temporary> (actual time=6.16..6.17 rows=16 loops=1) -> Aggregate using temporary table (actual time=6.16..6.16 rows=16 loops=1) -> Nested loop inner join (cost=575 rows=3693) (actual time=0.0776..4.35 rows=3476 loops=1) -> Table scan on co (cost=1.95 rows=17) (actual time=0.0233..0.0319 rows=16 loops=1) -> Index lookup on pr using countryId (countryId=co.countryId) (cost=13.3 rows=217) (actual time=0.0275..0.258 rows=217 loops=16) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(countryTotal.total) (cost=705..705 rows=1) (actual time=3.99..3.99 rows=1 loops=1) -> Table scan on countryTotal (cost=701..704 rows=16) (actual time=3.98..3.99 rows=16 loops=1) -> Materialize (cost=701..701 rows=16) (actual time=3.98..3.98 rows=16 loops=1) -> Group aggregate: sum(ProductionRecords.`value`) (cost=699 rows=16) (actual time=0.289..3.97 rows=16 loops=1) -> Index scan on ProductionRecords using countryId (cost=352 rows=3476) (actual time=0.139..3.8 rows=3476 loops=1) -> Filter: ('sum(ti.`value`)' > (select #5)) (actual time=2.54..2.54 rows=6 loops=1) -> Table scan on <temporary> (actual time=1.54..1.54 rows=16 loops=1) -> Aggregate using temporary table (actual time=1.53..1.53 rows=16 loops=1) -> Nested loop inner join (cost=165 rows=989) (actual time=0.0376..1.08 rows=931 loops=1) -> Table scan on co (cost=1.95 rows=17) (actual time=0.0064..0.0111 rows=16 loops=1) -> Index lookup on ti using countryId (countryId=co.countryId) (cost=4.09 rows=58.2) (actual time=0.0503..0.0635 rows=58.2 loops=16) -> Select #5 (subquery in condition; run only once) -> Aggregate: avg(countryTotal.total) (cost=193..193 rows=1) (actual time=0.996..0.996 rows=1 loops=1) -> Table scan on countryTotal (cost=189..192 rows=16) (actual time=0.994..0.995 rows=16 loops=1) -> Materialize (cost=189..189 rows=16) (actual time=0.993..0.993 rows=16 loops=1) -> Group aggregate: sum(TradeIndicators.`value`) (cost=187 rows=16) (actual time=0.156..0.988 rows=16 loops=1) -> Index scan on TradeIndicators using countryId (cost=94.4 rows=931) (actual time=0.153..0.941 rows=931 loops=1)
```

Query 1 with index on productionRecords.value;

```
EXPLAIN
-> Sort: countryName (cost=2.6..2.6 rows=0) (actual time=13.7..13.7 rows=9 loops=1) -> Table scan on <union temporary> (cost=2.5..2.5 rows=0) (actual time=13.7..13.7 rows=9 loops=1) -> Union materialize with deduplication (cost=0..0 rows=0) (actual time=13.7..13.7 rows=9 loops=1) -> Filter: ('sum(pr.`value`)' > (select #2)) (actual time=11.1..11.1 rows=3 loops=1) -> Table scan on <temporary> (actual time=7.1..7.1 rows=16 loops=1) -> Aggregate using temporary table (actual time=7.1..7.1 rows=16 loops=1) -> Nested loop inner join (cost=575 rows=3693) (actual time=0.0923..5.08 rows=3476 loops=1) -> Table scan on co (cost=1.95 rows=17) (actual time=0.0287..0.0376 rows=16 loops=1) -> Index lookup on pr using country (countryId=co.countryId) (cost=13.3 rows=217) (actual time=0.0295..0.3 rows=217 loops=16) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(countryTotal.total) (cost=705..705 rows=1) (actual time=4..4 rows=1 loops=1) -> Table scan on countryTotal (cost=701..704 rows=16) (actual time=3.99..3.99 rows=16 loops=1) -> Materialize (cost=701..701 rows=16) (actual time=3.99..3.99 rows=16 loops=1) -> Group aggregate: sum(ProductionRecords.`value`) (cost=699 rows=16) (actual time=0.291..3.97 rows=16 loops=1) -> Index scan on ProductionRecords using country (cost=352 rows=3476) (actual time=0.141..3.81 rows=3476 loops=1) -> Filter: ('sum(ti.`value`)' > (select #5)) (actual time=2.52..2.53 rows=6 loops=1) -> Table scan on <temporary> (actual time=1.52..1.52 rows=16 loops=1) -> Aggregate using temporary table (actual time=1.51..1.51 rows=16 loops=1) -> Nested loop inner join (cost=165 rows=989) (actual time=0.0403..1.07 rows=931 loops=1) -> Table scan on co (cost=1.95 rows=17) (actual time=0.00755..0.0123 rows=16 loops=1) -> Index lookup on ti using countryId (countryId=co.countryId) (cost=4.09 rows=58.2) (actual time=0.0501..0.0627 rows=58.2 loops=16) -> Select #5 (subquery in condition; run only once) -> Aggregate: avg(countryTotal.total) (cost=193..193 rows=1) (actual time=0.994..0.994 rows=1 loops=1) -> Table scan on countryTotal (cost=189..192 rows=16) (actual time=0.991..0.992 rows=16 loops=1) -> Materialize (cost=189..189 rows=16) (actual time=0.99..0.99 rows=16 loops=1) -> Group aggregate: sum(TradeIndicators.`value`) (cost=187 rows=16) (actual time=0.153..0.985 rows=16 loops=1) -> Index scan on TradeIndicators using countryId (cost=94.4 rows=931) (actual time=0.15..0.937 rows=931 loops=1)
```

No improvement or degradation. The values are not part of key and they are unique, using this an index will not improve performance beyond the default indices.

Query 1 with index on tradeIndicators.value;

```
EXPLAIN
-> Sort: countryName (cost=2.6..2.6 rows=0) (actual time=13.2..13.2 rows=9 loops=1) -> Table scan on <union temporary> (cost=2.5..2.5 rows=0) (actual time=13.2..13.2 rows=9 loops=1) -> Union materialize with deduplication (cost=0..0 rows=0) (actual time=13.2..13.2 rows=9 loops=1) -> Filter: ('sum(pr.`value`)' > (select #2)) (actual time=10.6..10.6 rows=3 loops=1) -> Table scan on <temporary> (actual time=6.64..6.64 rows=16 loops=1) -> Aggregate using temporary table (actual time=6.64..6.64 rows=16 loops=1) -> Nested loop inner join (cost=575 rows=3693) (actual time=0.094..4.71 rows=3476 loops=1) -> Table scan on co (cost=1.95 rows=17) (actual time=0.0271..0.0361 rows=16 loops=1) -> Index lookup on pr using country (countryId=co.countryId) (cost=13.3 rows=217) (actual time=0.0275..0.279 rows=217 loops=16) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(countryTotal.total) (cost=705..705 rows=1) (actual time=3.95..3.95 rows=1 loops=1) -> Table scan on countryTotal (cost=701..704 rows=16) (actual time=3.94..3.95 rows=16 loops=1) -> Materialize (cost=701..701 rows=16) (actual time=3.94..3.94 rows=16 loops=1) -> Group aggregate: sum(ProductionRecords.`value`) (cost=699 rows=16) (actual time=0.295..3.93 rows=16 loops=1) -> Index scan on ProductionRecords using country (cost=352 rows=3476) (actual time=0.145..3.77 rows=3476 loops=1) -> Filter: ('sum(ti.`value`)' > (select #5)) (actual time=2.55..2.55 rows=6 loops=1) -> Table scan on <temporary> (actual time=1.52..1.52 rows=16 loops=1) -> Aggregate using temporary table (actual time=1.52..1.52 rows=16 loops=1) -> Nested loop inner join (cost=165 rows=989) (actual time=0.0364..1.07 rows=931 loops=1) -> Table scan on co (cost=1.95 rows=17) (actual time=0.00683..0.0119 rows=16 loops=1) -> Index lookup on ti using countryId (countryId=co.countryId) (cost=4.09 rows=58.2) (actual time=0.0503..0.0623 rows=58.2 loops=16) -> Select #5 (subquery in condition; run only once) -> Aggregate: avg(countryTotal.total) (cost=193..193 rows=1) (actual time=1..1 rows=1 loops=1) -> Table scan on countryTotal (cost=189..192 rows=16) (actual time=1..1 rows=16 loops=1) -> Materialize (cost=189..189 rows=16) (actual time=1..1 rows=16 loops=1) -> Group aggregate: sum(TradeIndicators.`value`) (cost=187 rows=16) (actual time=0.161..0.996 rows=16 loops=1) -> Index scan on TradeIndicators using countryId (cost=94.4 rows=931) (actual time=0.157..0.951 rows=931 loops=1)
```

No improvements or degradation as expected from the same results as above.

This query relies on countryId for grouping, and adding values to compute averages. Therefore, the best index is on countryId for both productionRecords and tradeIndicators tables. The countryId is foreign key for both tables, so the engine automatically created the indices by default.

Query 2 Indexing

This is the baseline EXPLAIN ANALYZE result before adding any new indexes for Query 2, which shows an initial execution cost of 1613

```
EXPLAIN
-> Limit: 15 row(s) (actual time=21.21 rows=15 loops=1) -> Sort: annual_aggregated_value DESC, limit input to 15 row(s) per chunk (actual time=21.21 rows=15 loops=1) -> Table scan on <temporary> (actual time=20.3.20.7 rows=3476 loops=1)
-> Aggregate using temporary table (actual time=20.3.20.3 rows=3476 loops=1) -> Nested loop inner join (cost=1613 rows=3476) (actual time=0.148.6.49 rows=3476 loops=1) -> Nested loop inner join (cost=396 rows=3476) (actual time=0.143.4.5 rows=3476 loops=1) -> Table scan on cr (cost=0.65 rows=4) (actual time=0.0217.0.0271 rows=4 loops=1) -> Filter: (pr.countryId is not null) (cost=33.7 rows=869) (actual time=0.102.1.07 rows=869 loops=4) -> Index lookup on pr using cropld (cropld=cr.cropld) (cost=33.7 rows=869) (actual time=0.102.1.01 rows=869 loops=4) -> Single-row index lookup on co using PRIMARY (countryId=pr.countryId) (cost=0.25 rows=1) (actual time=417e-6.439e-6 rows=1 loops=3476)
```

This Index tests whether adding a covering index to the correlated subquery's target table improves Query 2. Used CREATE INDEX idx_ti_country_year_value ON TradeIndicators(countryId, year, value);

```
EXPLAIN
-> Limit: 15 row(s) (actual time=19.3.19.3 rows=15 loops=1) -> Sort: annual_aggregated_value DESC, limit input to 15 row(s) per chunk (actual time=19.3.19.3 rows=15 loops=1) -> Table scan on <temporary> (actual time=18.4.18.9 rows=3476 loops=1) -> Aggregate using temporary table (actual time=18.4.18.5 rows=3476 loops=1) -> Nested loop inner join (cost=1400 rows=5765) (actual time=0.0531.4.84 rows=3476 loops=1) -> Inner hash join (no condition) (cost=7.7 rows=68) (actual time=0.0215.0.0421 rows=64 loops=1) -> Table scan on co (cost=0.488 rows=17) (actual time=0.00367.0.0155 rows=16 loops=1) -> Hash -> Table scan on cr (cost=0.65 rows=4) (actual time=0.00914.0.0111 rows=4 loops=1) -> Index lookup on pr using idx_pr_grouping (countryId=co.countryId, cropld=cr.cropld) (cost=12.1 rows=84.8) (actual time=0.00931.0.0715 rows=54.3 loops=64)
```

This design tests whether adding a composite index to the main table's grouping and join columns improves Query 2. Used "CREATE INDEX idx_pr_grouping ON ProductionRecords(countryId, cropld, element, year);"

```
EXPLAIN
-> Limit: 15 row(s) (actual time=19.2.19.2 rows=15 loops=1) -> Sort: annual_aggregated_value DESC, limit input to 15 row(s) per chunk (actual time=19.2.19.2 rows=15 loops=1) -> Table scan on <temporary> (actual time=18.4.18.9 rows=3476 loops=1) -> Aggregate using temporary table (actual time=18.4.18.5 rows=3476 loops=1) -> Nested loop inner join (cost=1400 rows=5765) (actual time=0.0544.4.82 rows=3476 loops=1) -> Inner hash join (no condition) (cost=7.7 rows=68) (actual time=0.0244.0.0436 rows=64 loops=1) -> Table scan on co (cost=0.488 rows=17) (actual time=0.00437.0.0155 rows=16 loops=1) -> Hash -> Table scan on cr (cost=0.65 rows=4) (actual time=0.0103.0.0121 rows=4 loops=1) -> Index lookup on pr using idx_pr_grouping (countryId=co.countryId, cropld=cr.cropld) (cost=12.1 rows=84.8) (actual time=0.00892.0.0714 rows=54.3 loops=64)
```

This design tests whether adding an index on a low-cardinality descriptive name column inside a reference lookup table improves Query 2. Used "CREATE INDEX idx_cr_name ON Crops(cropName);"

```
EXPLAIN
-> Limit: 15 row(s) (actual time=21.2.21.2 rows=15 loops=1) -> Sort: annual_aggregated_value DESC, limit input to 15 row(s) per chunk (actual time=21.2.21.2 rows=15 loops=1) -> Table scan on <temporary> (actual time=20.4.20.8 rows=3476 loops=1) -> Aggregate using temporary table (actual time=20.4.20.4 rows=3476 loops=1) -> Nested loop inner join (cost=1613 rows=3476) (actual time=0.122.6.5 rows=3476 loops=1) -> Nested loop inner join (cost=396 rows=3476) (actual time=0.118.4.55 rows=3476 loops=1) -> Covering index scan on cr using idx_cr_name (cost=0.65 rows=4) (actual time=0.00873.0.0133 rows=4 loops=1) -> Filter: (pr.countryId is not null) (cost=33.7 rows=869) (actual time=0.0969.1.08 rows=869 loops=4) -> Index lookup on pr using cropld (cropld=cr.cropld) (cost=33.7 rows=869) (actual time=0.0966.1.03 rows=869 loops=4) -> Single-row index lookup on co using PRIMARY (countryId=pr.countryId) (cost=0.25 rows=1) (actual time=419e-6.439e-6 rows=1 loops=3476)
```

For Query 2, the default EXPLAIN ANALYZE plan yielded a root execution cost of 1613 due to scanning data pages for the inner subquery. We first tested an index on TradeIndicators(countryId, year, value). This produced a major improvement, significantly dropping the processing cost because it allowed a fast covering index lookup instead of scanning more rows from TradeIndicators.

Next, we tested a composite index on ProductionRecords(countryId, cropld, element, year) to optimize the heavy GROUP BY workload. This design brought a minor cost reduction from 1613 down to 1400. The improvement was relatively small compared to fixing the correlated subquery bottleneck.

Finally, we tested an index on Crops(cropName) to try to speed up the multi-table join sequence. This design did not produce any noticeable improvement or degradation, as the query planner opted to ignore the index entirely because a full table scan on the small reference table is already fast. Therefore, the final selected index for Query 2 is TradeIndicators(countryId, year, value), because it provides the largest cost reduction while avoiding unnecessary extra indexes on ProductionRecords.

Query 3 Indexing

This is the baseline EXPLAIN ANALYZE result before adding any new indexes for Query 3.

```
EXPLAIN
--> Limit: 15 row(s) (cost=3.61..3.61 rows=0) (actual time=91.5..91.5 rows=15 loops=1) --> Sort: ranked_country_years.total_production DESC, limit input to 15 row(s) per chunk (cost=3.61..3.61 rows=0)
(actual time=91.5..91.5 rows=15 loops=1) --> Index lookup on ranked_country_years using <auto_key0> (production_rank=1) (cost=0.35..3.51 rows=10) (actual time=91.5..91.5 rows=16 loops=1) -->
Materialize CTE ranked_country_years (cost=0..0 rows=0) (actual time=91.5..91.5 rows=518 loops=1) --> Window aggregate: row_number() OVER (PARTITION BY country_year_summary.countryName
ORDER BY country_year_summary.total_production desc) (cost=0 rows=0) (actual time=91.1..91.3 rows=518 loops=1) --> Sort: country_year_summary.countryName,
country_year_summary.total_production DESC (cost=2.6..2.6 rows=0) (actual time=91.1..91.1 rows=518 loops=1) --> Table scan on country_year_summary (cost=2.5..2.5 rows=0) (actual time=90.9..90.9
rows=518 loops=1) --> Materialize CTE country_year_summary if needed (cost=0..0 rows=0) (actual time=90.9..90.9 rows=518 loops=1) --> Group aggregate: sum(ProductionRecords.'value'),
avg(TradeIndicators.'value'), count(distinct ProductionRecords.cropld) (actual time=90.3..90.8 rows=518 loops=1) --> Sort: co.countryId, co.countryName, pr.'year' (actual time=90.3..90.4 rows=1633
loops=1) --> Stream results (cost=4182 rows=2247) (actual time=0.216..89.6 rows=1633 loops=1) --> Nested loop inner join (cost=4182 rows=2247) (actual time=0.213..89.2 rows=1633 loops=1) --> Nested
loop inner join (cost=487 rows=386) (actual time=0.0764..2.46 rows=1349 loops=1) --> Filter: ((pr.element like '%Production%') and (pr.countryId is not null)) (cost=352 rows=386) (actual time=0.0683..1.66
rows=1349 loops=1) --> Table scan on pr (cost=352 rows=3476) (actual time=0.0635..1.03 rows=3476 loops=1) --> Single-row index lookup on co using PRIMARY (countryId=pr.countryId) (cost=0.25
rows=1) (actual time=451e-6..472e-6 rows=1 loops=1349) --> Filter: (ti.'year' = pr.'year') (cost=3.75 rows=5.82) (actual time=0.0516..0.0641 rows=1.21 loops=1349) --> Index lookup on ti using countryId
(countryId=pr.countryId) (cost=3.75 rows=58.2) (actual time=0.0497..0.0617 rows=57 loops=1349) --> Select #2 (subquery in projection; run only once) --> Aggregate:
avg(country_year_summary.total_production) (cost=2.5..2.5 rows=1) (actual time=0.0777..0.0778 rows=1 loops=1) --> Table scan on country_year_summary (cost=2.5..2.5 rows=0) (actual
time=0.00466..0.053 rows=518 loops=1) --> Materialize CTE country_year_summary if needed (query plan printed elsewhere) (cost=0..0 rows=0) (never executed)
```

This design tests whether indexing the ProductionRecords filter, join, and grouping columns improves Query 3. Used “CREATE INDEX idx_pr_element_country_year ON ProductionRecords(element, countryId, year);”

```
EXPLAIN
--> Limit: 15 row(s) (cost=3.61..3.61 rows=0) (actual time=94.8..94.8 rows=15 loops=1) --> Sort: ranked_country_years.total_production DESC, limit input to 15 row(s) per chunk (cost=3.61..3.61 rows=0)
(actual time=94.8..94.8 rows=15 loops=1) --> Index lookup on ranked_country_years using <auto_key0> (production_rank=1) (cost=0.35..3.51 rows=10) (actual time=94.8..94.8 rows=16 loops=1) -->
Materialize CTE ranked_country_years (cost=0..0 rows=0) (actual time=94.8..94.8 rows=518 loops=1) --> Window aggregate: row_number() OVER (PARTITION BY country_year_summary.countryName
ORDER BY country_year_summary.total_production desc) (cost=0 rows=0) (actual time=94.4..94.5 rows=518 loops=1) --> Sort: country_year_summary.countryName,
country_year_summary.total_production DESC (cost=2.6..2.6 rows=0) (actual time=94.4..94.4 rows=518 loops=1) --> Table scan on country_year_summary (cost=2.5..2.5 rows=0) (actual time=94.2..94.2
rows=518 loops=1) --> Materialize CTE country_year_summary if needed (cost=0..0 rows=0) (actual time=94.2..94.2 rows=518 loops=1) --> Group aggregate: sum(ProductionRecords.'value'),
avg(TradeIndicators.'value'), count(distinct ProductionRecords.cropld) (actual time=93.6..94 rows=518 loops=1) --> Sort: co.countryId, co.countryName, pr.'year' (actual time=93.6..93.7 rows=1633
loops=1) --> Stream results (cost=4182 rows=2247) (actual time=0.201..92.8 rows=1633 loops=1) --> Nested loop inner join (cost=4182 rows=2247) (actual time=0.198..92.4 rows=1633 loops=1) --> Nested
loop inner join (cost=487 rows=386) (actual time=0.0646..2.65 rows=1349 loops=1) --> Filter: ((pr.element like '%Production%') and (pr.countryId is not null)) (cost=352 rows=386) (actual time=0.0557..1.8
rows=1349 loops=1) --> Table scan on pr (cost=352 rows=3476) (actual time=0.0507..1.12 rows=3476 loops=1) --> Single-row index lookup on co using PRIMARY (countryId=pr.countryId) (cost=0.25
rows=1) (actual time=489e-6..511e-6 rows=1 loops=1349) --> Filter: (ti.'year' = pr.'year') (cost=3.75 rows=5.82) (actual time=0.0532..0.0664 rows=1.21 loops=1349) --> Index lookup on ti using countryId
(countryId=pr.countryId) (cost=3.75 rows=58.2) (actual time=0.0512..0.0637 rows=57 loops=1349) --> Select #2 (subquery in projection; run only once) --> Aggregate:
avg(country_year_summary.total_production) (cost=2.5..2.5 rows=1) (actual time=0.0752..0.0752 rows=1 loops=1) --> Table scan on country_year_summary (cost=2.5..2.5 rows=0) (actual
time=0.00473..0.0516 rows=518 loops=1) --> Materialize CTE country_year_summary if needed (query plan printed elsewhere) (cost=0..0 rows=0) (never executed)
```

This design tests whether indexing the TradeIndicators join columns and aggregate value improves Query 3. Used “CREATE INDEX idx_ti_country_year_value ON TradeIndicators(countryId, year, value);”

```
EXPLAIN
--> Limit: 15 row(s) (cost=3.61..3.61 rows=0) (actual time=7.73..7.73 rows=15 loops=1) --> Sort: ranked_country_years.total_production DESC, limit input to 15 row(s) per chunk (cost=3.61..3.61 rows=0)
(actual time=7.73..7.73 rows=15 loops=1) --> Index lookup on ranked_country_years using <auto_key0> (production_rank=1) (cost=0.35..3.51 rows=10) (actual time=7.72..7.73 rows=16 loops=1) -->
Materialize CTE ranked_country_years (cost=0..0 rows=0) (actual time=7.72..7.72 rows=518 loops=1) --> Window aggregate: row_number() OVER (PARTITION BY country_year_summary.countryName
ORDER BY country_year_summary.total_production desc) (cost=0 rows=0) (actual time=7.3..7.5 rows=518 loops=1) --> Sort: country_year_summary.countryName, country_year_summary.total_production
DESC (cost=2.6..2.6 rows=0) (actual time=7.3..7.33 rows=518 loops=1) --> Table scan on country_year_summary (cost=2.5..2.5 rows=0) (actual time=7.13..7.18 rows=518 loops=1) --> Materialize CTE
country_year_summary if needed (cost=0..0 rows=0) (actual time=7.13..7.13 rows=518 loops=1) --> Group aggregate: sum(ProductionRecords.'value'), avg(TradeIndicators.'value'), count(distinct
ProductionRecords.cropld) (actual time=6.52..6.99 rows=518 loops=1) --> Sort: co.countryId, co.countryName, pr.'year' (actual time=6.51..6.59 rows=1633 loops=1) --> Stream results (cost=775 rows=471)
(actual time=0.123..5.83 rows=1633 loops=1) --> Nested loop inner join (cost=775 rows=471) (actual time=0.119..5.44 rows=1633 loops=1) --> Nested loop inner join (cost=487 rows=386) (actual
time=0.0934..2.7 rows=1349 loops=1) --> Filter: ((pr.element like '%Production%') and (pr.countryId is not null) and (pr.'year' is not null)) (cost=352 rows=386) (actual time=0.0811..1.85 rows=1349 loops=1)
--> Table scan on pr (cost=352 rows=3476) (actual time=0.0758..1.14 rows=3476 loops=1) --> Single-row index lookup on co using PRIMARY (countryId=pr.countryId) (cost=0.25 rows=1) (actual time=486e-
6..508e-6 rows=1 loops=1349) --> Covering index lookup on ti using idx_ti_country_year_value (countryId=pr.countryId, year=pr.'year') (cost=0.626 rows=1.22) (actual time=0.00146..0.00186 rows=1.21
loops=1349) --> Select #2 (subquery in projection; run only once) --> Aggregate: avg(country_year_summary.total_production) (cost=2.5..2.5 rows=1) (actual time=0.0747..0.0747 rows=1 loops=1) --> Table
scan on country_year_summary (cost=2.5..2.5 rows=0) (actual time=0.00325..0.0501 rows=518 loops=1) --> Materialize CTE country_year_summary if needed (query plan printed elsewhere) (cost=0..0
rows=0) (never executed)
```

This design tests whether combining indexes on both major tables improves Query 3 beyond the individual indexes. Used “CREATE INDEX idx_pr_element_country_year ON ProductionRecords(element, countryId,

year);", "CREATE INDEX idx_ti_country_year_value ON TradeIndicators(countryId, year, value);", "CREATE INDEX idx_pr_country_year_crop_value ON ProductionRecords(countryId, year, cropId, value);"

```

EXPLAIN
-> Limit: 15 row(s) (cost=3.61..3.61 rows=0) (actual time=6.9..6.91 rows=15 loops=1) -> Sort: ranked_country_years.total_production DESC, limit input to 15 row(s) per chunk (cost=3.61..3.61 rows=0)
(actual time=6.9..6.9 rows=15 loops=1) -> Index lookup on ranked_country_years using <auto_key0> (production_rank=1) (cost=0.35..3.51 rows=10) (actual time=6.89..6.9 rows=16 loops=1) -> Materialize
CTE ranked_country_years (cost=0..0 rows=0) (actual time=6.89..6.89 rows=518 loops=1) -> Window aggregate: row_number() OVER (PARTITION BY country_year_summary.countryName ORDER BY
country_year_summary.total_production desc) (cost=0 rows=0) (actual time=6.5..6.67 rows=518 loops=1) -> Sort: country_year_summary.countryName, country_year_summary.total_production DESC
(cost=2.6..2.6 rows=0) (actual time=6.49..6.52 rows=518 loops=1) -> Table scan on country_year_summary (cost=2.5..2.5 rows=0) (actual time=6.32..6.38 rows=518 loops=1) -> Materialize CTE
country_year_summary if needed (cost=0..0 rows=0) (actual time=6.32..6.32 rows=518 loops=1) -> Group aggregate: sum(ProductionRecords.`value`), avg(TradeIndicators.`value`), count(distinct
ProductionRecords.cropId) (actual time=5.74..6.2 rows=518 loops=1) -> Sort: co.countryId, co.countryName, pr.`year` (actual time=5.73..5.82 rows=1633 loops=1) -> Stream results (cost=775 rows=471)
(actual time=0.0823..5.06 rows=1633 loops=1) -> Nested loop inner join (cost=775 rows=471) (actual time=0.0798..4.72 rows=1633 loops=1) -> Nested loop inner join (cost=487 rows=386) (actual
time=0.0648..2.36 rows=1349 loops=1) -> Filter: ((pr.element like '%Production%') and (pr.countryId is not null) and (pr.`year` is not null)) (cost=352 rows=386) (actual time=0.0561..1.61 rows=1349
loops=1) -> Table scan on pr (cost=352 rows=3476) (actual time=0.052..0.996 rows=3476 loops=1) -> Single-row index lookup on co using PRIMARY (countryId=pr.countryId) (cost=0.25 rows=1) (actual
time=417e-6..437e-6 rows=1 loops=1349) -> Covering index lookup on ti using idx_ti_country_year_value (countryId=pr.countryId, year=pr.`year`) (cost=0.626 rows=1.22) (actual time=0.00124..0.00159
rows=1.21 loops=1349) -> Select #2 (subquery in projection; run only once) -> Aggregate: avg(country_year_summary.total_production) (cost=2.5..2.5 rows=1) (actual time=0.0747..0.0747 rows=1
loops=1) -> Table scan on country_year_summary (cost=2.5..2.5 rows=0) (actual time=0.00235..0.0504 rows=518 loops=1) -> Materialize CTE country_year_summary if needed (query plan printed
elsewhere) (cost=0..0 rows=0) (never executed)

```

For Query 3, the default EXPLAIN ANALYZE plan had a main join cost of approximately 4182. We first tested an index on ProductionRecords(element, countryId, year), but this did not meaningfully improve the cost. This is likely because the query uses LIKE '%Production%', which has a leading wildcard and limits the usefulness of an index on element. The query also still needs to scan and aggregate many ProductionRecords rows.

Next, we tested an index on TradeIndicators(countryId, year, value). This produced a major improvement, reducing the main join cost from approximately 4182 to approximately 775. This improvement occurred because Query 3 joins TradeIndicators using both countryId and year, and the index allowed MySQL to perform a covering index lookup instead of scanning more rows from TradeIndicators.

Finally, we tested a combined index design using indexes on both ProductionRecords and TradeIndicators. The combined design did not significantly improve the cost beyond the TradeIndicators(countryId, year, value) index alone. Therefore, the final selected index for Query 3 is TradeIndicators(countryId, year, value), because it provides the largest cost reduction while avoiding unnecessary extra indexes on ProductionRecords.